

# Word Prediction Using a Neural Language Model

## Instructions

1. Follow the instructions in each question carefully.
2. A Jupyter notebook in PDF format along with output for each cell is expected.
3. Any assignment submitted using other python IDEs are not considered for grading.
4. Incorrect Assignment Set submitted will not be considered.
5. Observation and inference have to be mentioned for each question. Techniques used without justification or inference will not be awarded marks.

## 1. Problem Statement

Your task is to implement a complete word-prediction pipeline. You will preprocess a text corpus, build a custom PyTorch dataset, design a character/word-level recurrent neural network, train it on text data, and use it to generate text auto-completions based on a user prompt.

### Objectives

- **Data Pipeline:** Tokenize a text corpus and build a mapping between words and unique integer IDs (Vocabulary).
- **Dataset & DataLoader:** Construct sliding window sequences of length  $N$  to serve as model inputs, with the  $(N + 1)$  th word as the target.
- **Model Architecture:** Implement an Embedding layer, an LSTM/RNN layer, and a Linear output layer.
- **Evaluation:** Calculate **Perplexity (PPL)** to evaluate your model's performance and implement a top- $k$  decoding strategy for text generation.

## 2. Dataset Specifications

You will be using a dataset PennTreebank

(<https://docs.pytorch.org/text/0.8.1/datasets.html#language-modeling>)

- **Vocabulary Size ( $V$ ):** Bound to the top 10,000 most frequent words. All other words must be replaced with an <unk> (unknown) token.
- **Special Tokens:** Include <pad> for padding and <unk> for out-of-vocabulary words.

## 3. Tasks and Mark Breakdown (Total: 12 Marks)

**Task 1: Implement the below tasks using Virtual Lab (1 Mark)**

**Task 2: Data Preprocessing & Vocabulary Building (3 Marks)**

1. Read train.txt and tokenize the text into lowercase words/tokens.

2. Build a vocabulary mapping `word_to_id` and `id_to_word`.
3. Implement a sliding window function to create your features ( $X$ ) and labels ( $y$ ).
  - o *Example:* If sequence length is 3, the sentence “the cat sat on the mat” yields:
    - $X: ["the", "cat", "sat"] \rightarrow y: "on"$
    - $X: ["cat", "sat", "on"] \rightarrow y: "the"$

### Task 3: Model Architecture Implementation (3 Marks)

Implement a neural network class `WordPredictorLSTM` using both character/word-level inheriting from `torch.nn.Module`. Your network must follow this structural flow:

- **Embedding Layer:** Maps word IDs to dense vectors of size  $D$  (`embedding_dim`).
- **Recurrent Layer:** An LSTM or GRU layer with hidden dimension  $H$  (`hidden_dim`) and  $L$  layers.
- **Linear Layer:** Maps the final hidden state to a vector of size  $V$  (Vocabulary Size) representing the unnormalized log probabilities (logits) for each word.

### Task 4: Model Training & Optimization (3 Marks)

1. Define your loss function using **Cross-Entropy Loss**. Ensure you ignore the index corresponding to the `<pad>` token if padding is used.
2. Train the model for a minimum of 10 epochs using the Adam optimizer.
3. Track and plot both **Training Loss** and **Validation Perplexity** per epoch.

**Note:** Perplexity is calculated as the exponential of the cross-entropy loss:  
$$PPL = e^{Loss}.$$

### Task 5: Inference & Text Generation (2 Marks)

Implement a function `predict_next_words(prompt, top_k=3)` that takes a string prompt, tokenizes it, passes it through the trained model, and outputs the top 3 most likely next words along with their probabilities using a softmax distribution over the final layer output.

**Justification of the output obtained for all the above tasks is mandatory**